

Guess, Check, Correct

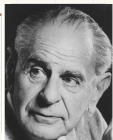
Foundations of AI

Carlos Cotrini

UZH CAS Gamechanger AI

26 September 2026

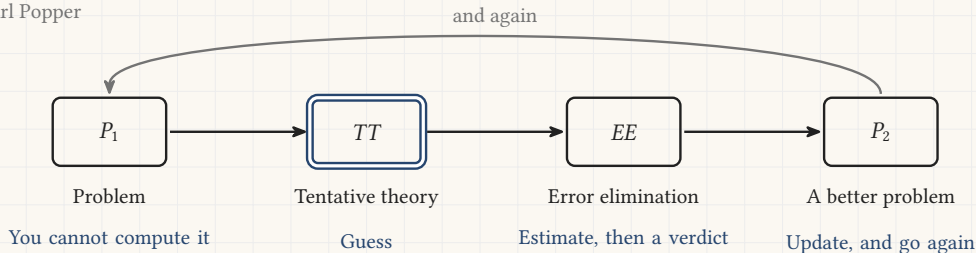
Popper, 1934: the tetradic schema



Karl Popper

We do not know: we can only guess.

Karl Popper, *The Logic of Scientific Discovery*, chapter 10, section 85 (1959).



This weekend is that diagram, and how it produces the AI applications you already use.

Schema after Karl Popper, *Objective Knowledge*, chapter 8 (1972).

Foundations of AI, UZH CAS Gamechanger AI, 26 September 2026

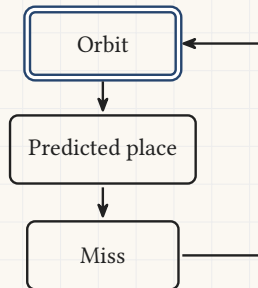
Astronomy: the recovery of Ceres, 1801



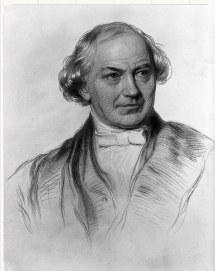
Carl Friedrich
Gauss, 1840.

C. A. Jensen

- **January 1801.** Giuseppe Piazzi finds a new object and tracks it for 41 days.
- **February 1801.** He loses it in the Sun's glare, with too few observations to fix an orbit.
- **Late 1801.** Carl Friedrich Gauss guesses an orbit, predicts, and corrects.
- **December 1801.** Franz Xaver von Zach finds it again.



Whewell, 1858: trying wrong guesses



William Whewell, stipple engraving.
Wellcome Collection, CC BY 4.0

To try **wrong guesses** is, with most persons, the only way to hit upon right ones.

The tendencies of our speculative nature ... perpetually show their vigour by **overshooting the mark**. They obtain something, by aiming at much more.

Most guesses are wrong, and a step that aims too far is how the right one gets found.

William Whewell, *Novum Organon Renovatum*, third edition (1858), book II, chapter V, page 79.

Computer science: Turing, 1950



Alan Turing, 29 March 1951.
Elliott and Fry

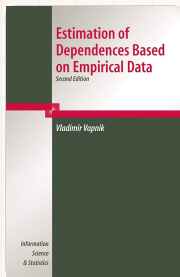
We cannot expect to find a good child-machine at the first attempt. One must experiment with teaching one such machine and see how well it learns. One can then **try another** and see if it is better or worse.

Now the learning process may be regarded as a **search for a form of behaviour** which will satisfy the teacher (or some other criterion).

The loop was written down as the plan for machine intelligence before there was a machine to run it on.

A. M. Turing, "Computing Machinery and Intelligence", *Mind* LIX(236), 1950, section 7.

The loop has a name, and the name is a book



Second edition, Springer, 2006

Vladimir Vapnik. The Russian original is 1979; Samuel Kotz's English translation is Springer, 1982.

The title is the whole claim. You are handed **empirical data**, and what you estimate from it is a **dependence**: the thing in the world that produced the numbers.

The book sets out **empirical risk minimisation**, which is the strategy underneath most of what the rest of this course will build.

Vladimir N. Vapnik, *Estimation of Dependences Based on Empirical Data*. Translated by Samuel Kotz. New York: Springer, 1982; second edition, Springer, 2006.

First-order approximation algorithms

Foundations of AI

Carlos Cotrini

UZH CAS Gamechanger AI

26 September 2026

1

Section 1

Two problems, done by hand

Try it: Square One

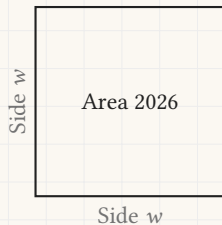
Switzerland, folded into a perfect square.

How long is its side?

<https://carloscotrini.github.io/FAI-PAGE/viz/square-root-show/>



Calculating square roots

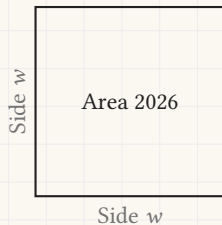


Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
-----	-------	------------------	--------

How long is the side?

Calculating square roots

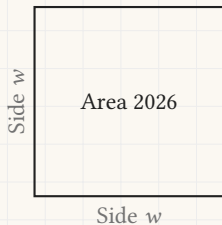


Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	+426	↑

How long is the side?

Calculating square roots

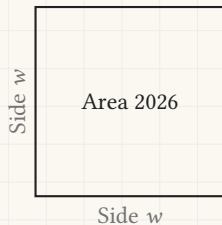


Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	+426	↑
50	2500	-474	↓

How long is the side?

Calculating square roots

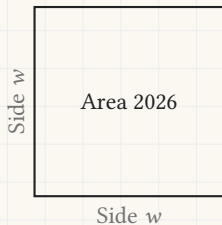


Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	+426	↑
50	2500	-474	↓
45	2025	+1	↑

How long is the side?

Calculating square roots

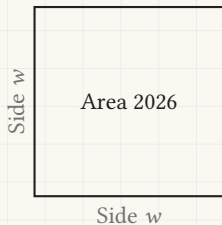


How long is the side?

Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	+426	↑
50	2500	-474	↓
45	2025	+1	↑
⋮	⋮	⋮	

Calculating square roots



How long is the side?

Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	+426	↑
50	2500	-474	↓
45	2025	+1	↑
⋮	⋮	⋮	

Nobody here computed a square root. Each row is a guess that was scored, and the score said which way to move.

The update rule

$$w := w$$

w	w^2	l
-----	-------	-----

w	w^2	l
-----	-------	-----

The update rule

$$w := w + \underbrace{l}_{\text{The verdict}}$$

w	w^2	l
-----	-------	-----

w	w^2	l
-----	-------	-----

The update rule

$$w := w + \underbrace{\alpha}_{\text{How far to move}} \underbrace{l}_{\text{The verdict}}$$

w	w^2	l
-----	-------	-----

w	w^2	l
-----	-------	-----

The update rule

$$w := w + \underbrace{\alpha}_{\text{How far to move}} \underbrace{l}_{\text{The verdict}}$$

w	w^2	l
40.000	1600.000	426.000
42.130	1774.937	251.063
43.385	1882.286	143.714
44.104	1945.153	80.847
44.508	1980.973	45.027
44.733	2001.064	24.936
44.858	2012.234	13.766
44.927	2018.414	7.586
44.965	2021.824	4.176
44.986	2023.702	2.298
44.997	2024.736	1.264
45.003	2025.305	0.695

w	w^2	l
-----	-------	-----

- Starting at $w = 40$, with $\alpha = 0.005$.

The update rule

$$w := w + \underbrace{\alpha}_{\text{How far to move}} \underbrace{l}_{\text{The verdict}}$$

w	w^2	l
40.000	1600.000	426.000
42.130	1774.937	251.063
43.385	1882.286	143.714
44.104	1945.153	80.847
44.508	1980.973	45.027
44.733	2001.064	24.936
44.858	2012.234	13.766
44.927	2018.414	7.586
44.965	2021.824	4.176
44.986	2023.702	2.298
44.997	2024.736	1.264
45.003	2025.305	0.695

w	w^2	l
45.007	2025.618	0.382
45.009	2025.790	0.210
45.010	2025.884	0.116
45.010	2025.936	0.064
45.011	2025.965	0.035
45.011	2025.981	0.019
45.011	2025.989	0.011
45.011	2025.994	0.006
45.011	2025.997	0.003
45.011	2025.998	0.002
45.011	2025.999	0.001

- Starting at $w = 40$, with $\alpha = 0.005$.
- The whole run, all 23 rounds of it.
- From the fifth row of this block, w has stopped moving and the verdict is still shrinking.

The update rule

$$w := w + \underbrace{\alpha}_{\text{How far to move}} \underbrace{l}_{\text{The verdict}}$$

w	w^2	l
40.000	1600.000	426.000
42.130	1774.937	251.063
43.385	1882.286	143.714
44.104	1945.153	80.847
44.508	1980.973	45.027
44.733	2001.064	24.936
44.858	2012.234	13.766
44.927	2018.414	7.586
44.965	2021.824	4.176
44.986	2023.702	2.298
44.997	2024.736	1.264
45.003	2025.305	0.695

w	w^2	l
45.007	2025.618	0.382
45.009	2025.790	0.210
45.010	2025.884	0.116
45.010	2025.936	0.064
45.011	2025.965	0.035
45.011	2025.981	0.019
45.011	2025.989	0.011
45.011	2025.994	0.006
45.011	2025.997	0.003
45.011	2025.998	0.002
45.011	2025.999	0.001

- Starting at $w = 40$, with $\alpha = 0.005$.
- The whole run, all 23 rounds of it.
- From the fifth row of this block, w has stopped moving and the verdict is still shrinking.
- $\sqrt{2026} = 45.011110$, and no square root was ever taken.

α is the step size. The letter ε is reserved: it is the stopping tolerance, and at 11:00 it becomes the width of a difference window.

Solving an equation

Find w with $w^3 - 3w^2 + 3w - 2 = 4$.

w	$f(w)$	$l = 4 - f(w)$	Update
-----	--------	----------------	--------

Solving an equation

Find w with $w^3 - 3w^2 + 3w - 2 = 4$.

w	$f(w)$	$l = 4 - f(w)$	Update
0	-2	+6	↑

Solving an equation

Find w with $w^3 - 3w^2 + 3w - 2 = 4$.

w	$f(w)$	$l = 4 - f(w)$	Update
0	-2	+6	↑
6	124	-120	↓

Solving an equation

Find w with $w^3 - 3w^2 + 3w - 2 = 4$.

w	$f(w)$	$l = 4 - f(w)$	Update
0	-2	+6	↑
6	124	-120	↓
2	0	+4	↑

Solving an equation

Find w with $w^3 - 3w^2 + 3w - 2 = 4$.

w	$f(w)$	$l = 4 - f(w)$	Update
0	-2	+6	↑
6	124	-120	↓
2	0	+4	↑
⋮	⋮	⋮	

The algorithm

```
def solve(f, target, w, alpha):  
    l = target - f(w)           # the verdict: how far off, and which way  
    while abs(l) > 0.001:       # not close enough yet  
        w = w + alpha * l      # the update: move that way  
        l = target - f(w)      # score the new guess  
    return w
```

The two problems, one call

The two problems of the last ten minutes are the same call, with a different f :

```
def square(w):  
    return w**2  
def cubic(w):  
    return w**3 - 3*w**2 + 3*w - 2  
solve(square, 2026, 40, 0.005)  
solve(cubic, 4, 0, 0.05)
```

2

Section 2

Iterative problem solving

Iterative problem solving

Iterative problem solving

Guess A value of w , however crude.

Guess

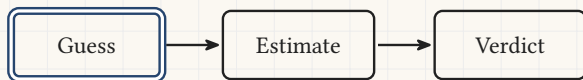
Iterative problem solving

Guess A value of w , however crude.

Estimate What the guess predicts.



Iterative problem solving

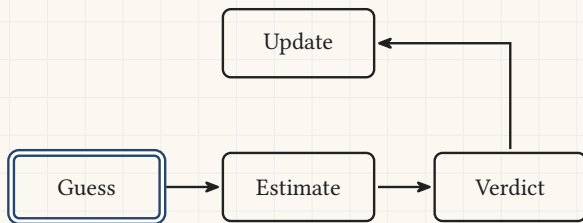


Guess A value of w , however crude.

Estimate What the guess predicts.

Verdict The signed miss l .

Iterative problem solving



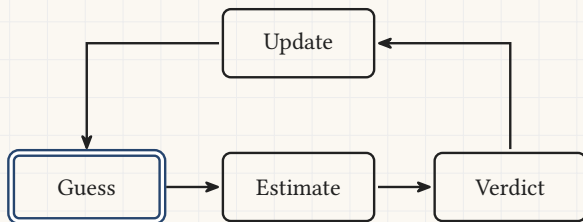
Guess A value of w , however crude.

Estimate What the guess predicts.

Verdict The signed miss l .

Update Move w the way the verdict says.

Iterative problem solving



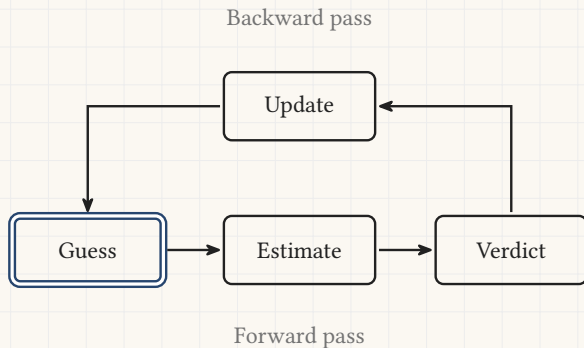
Guess A value of w , however crude.

Estimate What the guess predicts.

Verdict The signed miss l .

Update Move w the way the verdict says.

Iterative problem solving



Guess A value of w , however crude.

Estimate What the guess predicts.

Verdict The signed miss l .

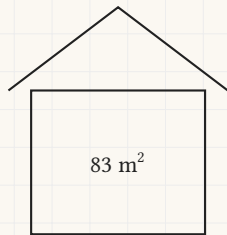
Update Move w the way the verdict says.

3

Section 3

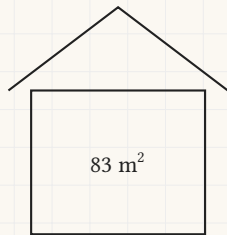
Estimating house prices

One house, one measurement



2 156 342 CHF

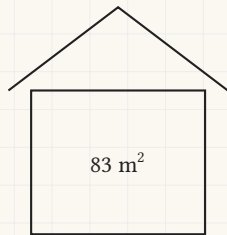
One house, one measurement



2 156 342 CHF

Data $x = 83$, the area. $y = 2\,156\,342$, the price.

One house, one measurement

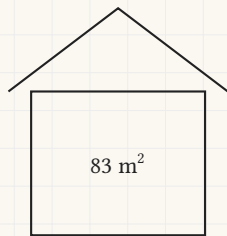


2 156 342 CHF

Data $x = 83$, the area. $y = 2\,156\,342$, the price.

Hypothesis Price is proportional to area, so the estimate is $\hat{y} = w x$.

One house, one measurement



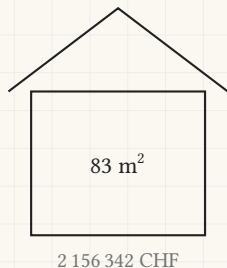
2 156 342 CHF

Data $x = 83$, the area. $y = 2\,156\,342$, the price.

Hypothesis Price is proportional to area, so the estimate is $\hat{y} = w x$.

Verdict The signed miss $l = y - \hat{y}$.

One house, one measurement



Data $x = 83$, the area. $y = 2\,156\,342$, the price.

Hypothesis Price is proportional to area, so the estimate is $\hat{y} = w x$.

Verdict The signed miss $l = y - \hat{y}$.

Problem Find the w that drives l to zero.

One house: the loop

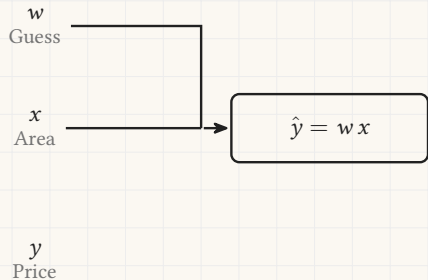
One house: the loop

w
Guess

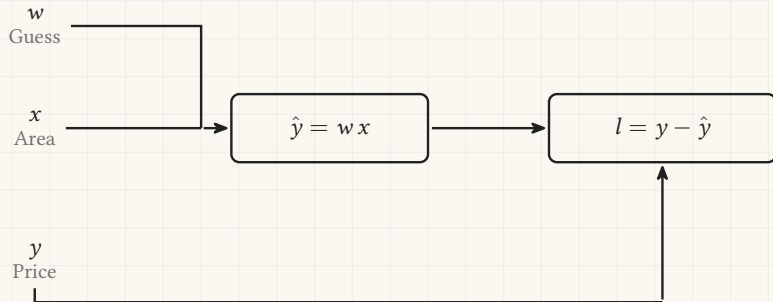
x
Area

y
Price

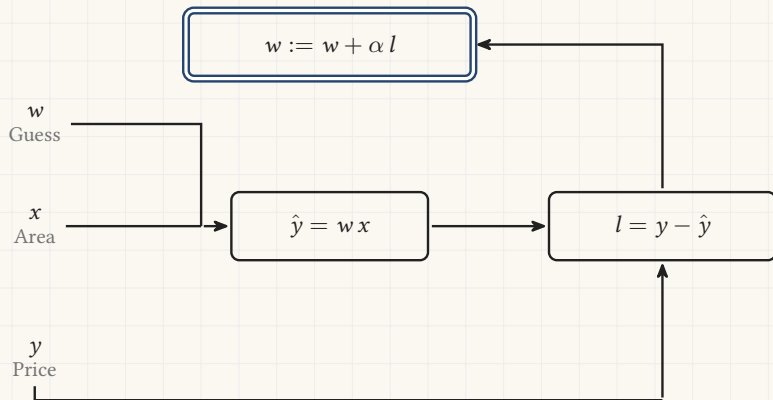
One house: the loop



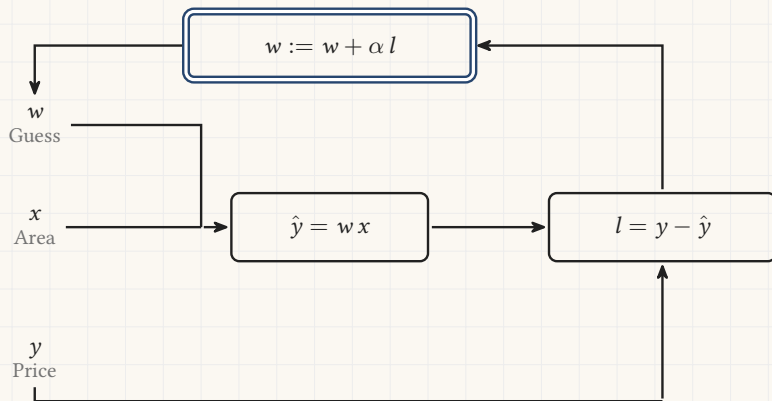
One house: the loop



One house: the loop



One house: the loop



One house: the algorithm

```
def fit(x, y, w, alpha):  
    l = y - w * x           # the verdict: y minus the estimate  
    while abs(l) > 0.5:      # more than half a franc out  
        w = w + alpha * l   # the update  
        l = y - w * x       # score the new guess  
    return w
```

One house: the algorithm

```
def fit(x, y, w, alpha):  
    l = y - w * x           # the verdict: y minus the estimate  
    while abs(l) > 0.5:      # more than half a franc out  
        w = w + alpha * l    # the update  
        l = y - w * x        # score the new guess  
    return w
```

On the house above:

```
fit(83, 2156342, 20000, 0.01)
```

One house: the algorithm

```
def fit(x, y, w, alpha):  
    l = y - w * x           # the verdict: y minus the estimate  
    while abs(l) > 0.5:     # more than half a franc out  
        w = w + alpha * l  # the update  
        l = y - w * x      # score the new guess  
    return w
```

On the house above:

```
fit(83, 2156342, 20000, 0.01)
```

Starting from a guess of 20 000 francs per square metre, this returns 25 980.02 after 8 rounds.

Many houses, one measurement

House	Area	Price
-------	------	-------

Many houses, one measurement

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113

House 1 is the house on the last slide. Houses 2 to 5 are synthetic, generated with a fixed seed.

Many houses, one measurement

Hypothesis The same one, row by row: $\hat{y}_i = w x_i$.

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113

House 1 is the house on the last slide. Houses 2 to 5 are synthetic, generated with a fixed seed.

Many houses, one measurement

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113

Hypothesis The same one, row by row: $\hat{y}_i = w x_i$.

Verdict One miss per row, $l_i = y_i - \hat{y}_i$.

House 1 is the house on the last slide. Houses 2 to 5 are synthetic, generated with a fixed seed.

Many houses, one measurement

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113

Hypothesis The same one, row by row: $\hat{y}_i = w x_i$.

Verdict One miss per row, $l_i = y_i - \hat{y}_i$.

Problem No single w empties every row. Drive the *average* verdict $\bar{l}$ to zero instead.

$$\bar{l} = \frac{l_1 + l_2 + \dots + l_n}{n}$$

House 1 is the house on the last slide. Houses 2 to 5 are synthetic, generated with a fixed seed.

Many houses: the loop

Many houses: the loop

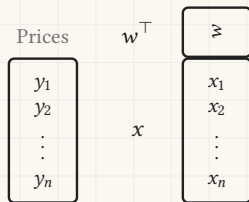
Prices

y_1
 y_2
 $\vdots$
 y_n

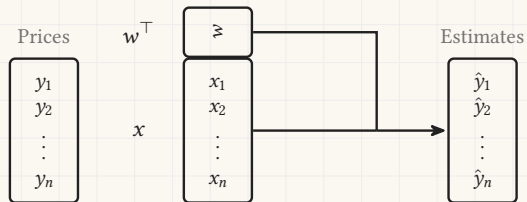
x

x_1
 x_2
 $\vdots$
 x_n

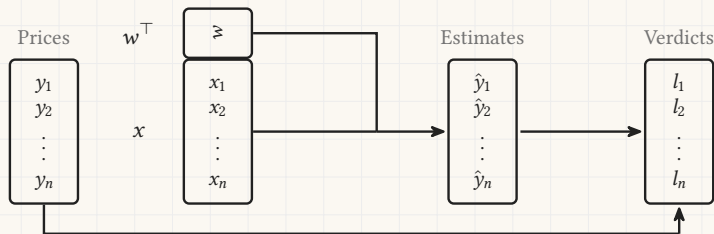
Many houses: the loop



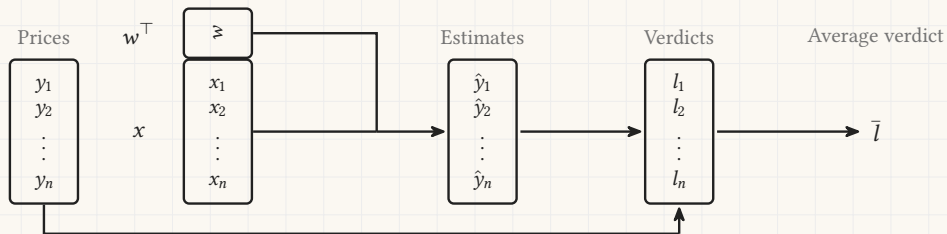
Many houses: the loop



Many houses: the loop

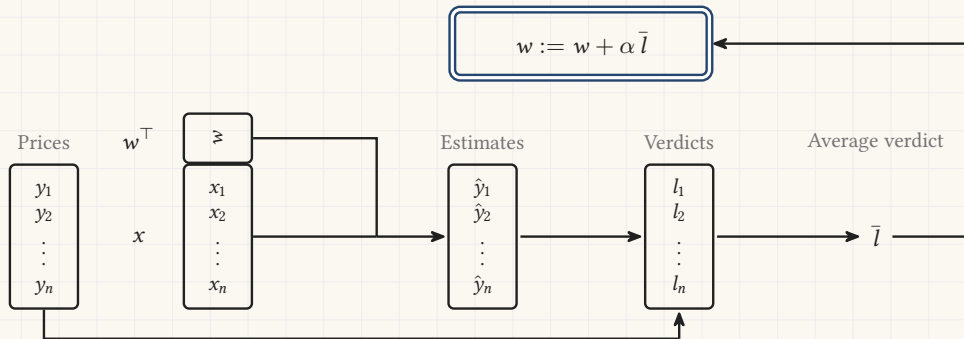


Many houses: the loop



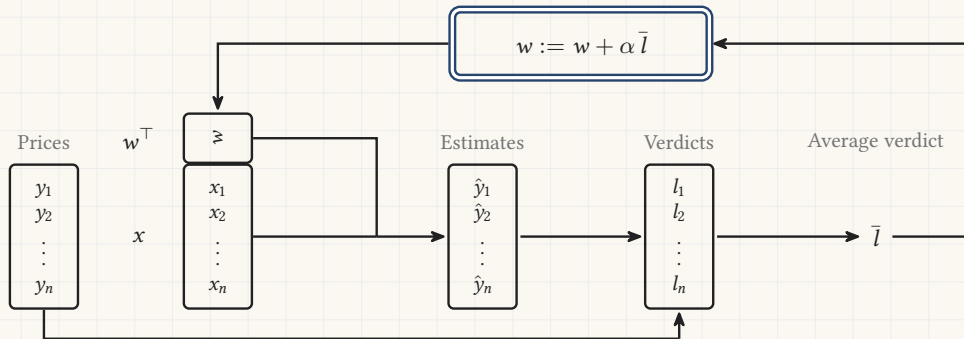
x holds the areas, and $\hat{y}_i = w x_i$, $l_i = y_i - \hat{y}_i$, $\bar{l} = (l_1 + \dots + l_n)/n$

Many houses: the loop



x holds the areas, and $\hat{y}_i = w x_i$, $l_i = y_i - \hat{y}_i$, $\bar{l} = (l_1 + \dots + l_n)/n$

Many houses: the loop



x holds the areas, and $\hat{y}_i = w x_i$, $l_i = y_i - \hat{y}_i$, $\bar{l} = (l_1 + \dots + l_n)/n$

Many houses: the algorithm

```
def verdict(x, y, w):           # the average miss over the rows
    total = 0
    for i in range(len(x)):
        total = total + (y[i] - w * x[i])
    return total / len(x)

def fit(x, y, w, alpha):
    l_bar = verdict(x, y, w)
    while abs(l_bar) > 0.5:
        w = w + alpha * l_bar
        l_bar = verdict(x, y, w)
    return w
```

Many measurements for each house

House	Area	Rooms	Age	Walk	Price

Many measurements for each house

House	Area	Rooms	Age	Walk	Price
1	83	3.5	12	4	2 156 342
2	62	2.5	41	11	1 461 635
3	104	4.5	28	8	2 650 166
4	128	5.5	6	15	3 184 899
5	145	6.5	55	6	3 651 113

Many measurements for each house

House	Area	Rooms	Age	Walk	Price
1	83	3.5	12	4	2 156 342
2	62	2.5	41	11	1 461 635
3	104	4.5	28	8	2 650 166
4	128	5.5	6	15	3 184 899
5	145	6.5	55	6	3 651 113

- Each house is now a list of d measurements, $x_i = (x_{i1}, x_{i2}, \dots, x_{id})$.

Many measurements for each house

House	Area	Rooms	Age	Walk	Price
1	83	3.5	12	4	2 156 342
2	62	2.5	41	11	1 461 635
3	104	4.5	28	8	2 650 166
4	128	5.5	6	15	3 184 899
5	145	6.5	55	6	3 651 113

- Each house is now a list of d measurements, $x_i = (x_{i1}, x_{i2}, \dots, x_{id})$.
- Each measurement keeps its own weight, so $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id}$.

Many measurements for each house

House	Area	Rooms	Age	Walk	Price
1	83	3.5	12	4	2 156 342
2	62	2.5	41	11	1 461 635
3	104	4.5	28	8	2 650 166
4	128	5.5	6	15	3 184 899
5	145	6.5	55	6	3 651 113

- Each house is now a list of d measurements, $x_i = (x_{i1}, x_{i2}, \dots, x_{id})$.
- Each measurement keeps its own weight, so $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id}$.
- One verdict, d unknowns. Pick one weight per round and move it.

Many measurements: the loop

Many measurements: the loop

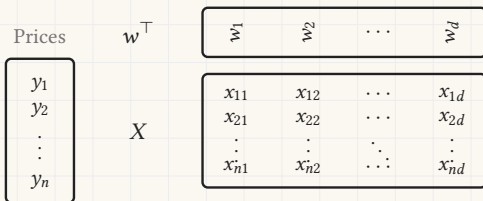
Prices

y_1
y_2
$\vdots$
y_n

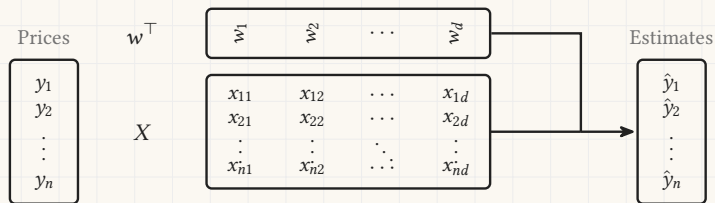
X

x_{11}	x_{12}	$\dots$	x_{1d}
x_{21}	x_{22}	$\dots$	x_{2d}
$\vdots$	$\vdots$	$\ddots$	$\vdots$
x_{n1}	x_{n2}	$\dots$	x_{nd}

Many measurements: the loop

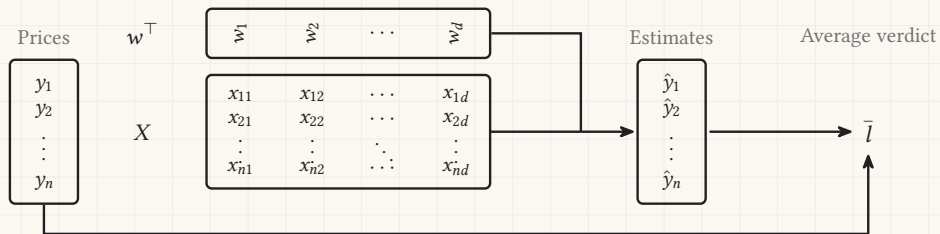


Many measurements: the loop



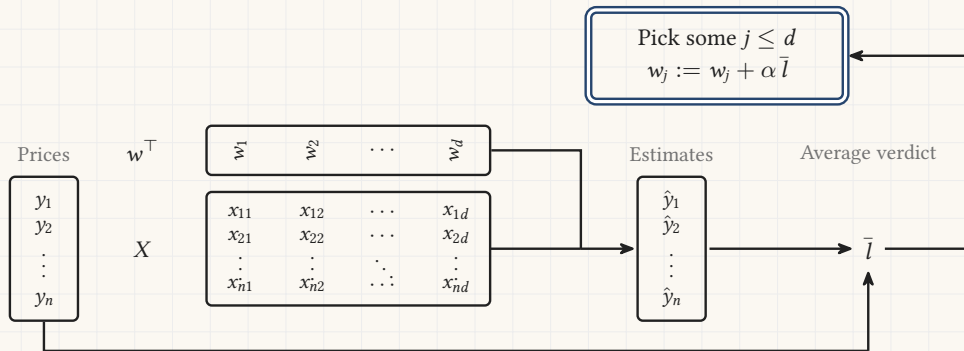
X has one row per house and one column per measurement, and $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id}$.

Many measurements: the loop



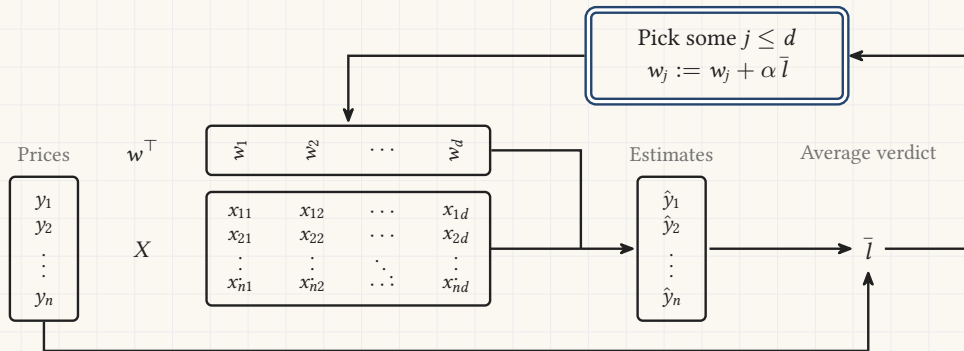
X has one row per house and one column per measurement, and $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id}$.

Many measurements: the loop



X has one row per house and one column per measurement, and $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id}$.

Many measurements: the loop



X has one row per house and one column per measurement, and $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id}$.